CENTRO UNIVERSITÁRIO PRESIDENTE ANTÔNIO CARLOS
UNIPAC Barbacena

Relatório Técnico

Consultor Inteligente de Carreiras em TI

Sistema Especialista baseado em regras
com Experta e Streamlit

Aluno(a): Marco Antônio da Silva Milagres
Disciplina: Inteligência Artificial
Professor(a): Felipe Carneiro
Repositório: https://github.com/cotonho/Consultor-de-carreiras-TI
Data: Outubro de 2026

Sumário

1 Visão Geral **2**

2 Requisitos da Atividade e Atendimento **2**

3 Tecnologias Utilizadas **3**

4 Engenharia de Conhecimento **3**

4.1 Fatos de entrada 3

4.2 Conflitos entre trilhas e como são resolvidos 4

5 Arquitetura e Implementação **4**

5.1 Estrutura do sistema 4

5.2 Base de regras 5

5.3 Pontuação, consolidação e desempate 6

6 Interface Web **6**

7 Validação **7**

7.1 Personas de teste 7

7.2 Cobertura de todo o espaço de respostas 7

8 Limitações e Melhorias **8**

9 Como Executar **8**

10 Conclusão **9**

1 Visão Geral

O Consultor Inteligente de Carreiras em TI é um Sistema Especialista baseado em regras (SE-ENTÃO) que entrevista o usuário e recomenda uma entre dez trilhas de carreira em Tecnologia da Informação. A decisão é acompanhada de uma justificativa transparente, que lista as regras disparadas e os pontos que cada uma atribuiu à trilha recomendada.

O sistema reúne 10 trilhas de carreira, 35 regras exttt@Rule, 5 dimensões de entrevista e foi validado com 10 personas, todas classificadas corretamente.

Resultado em uma frase. O sistema acerta todas as 10 personas de teste, mas uma varredura das 2.700 combinações possíveis de respostas mostrou que **642 (23,8%) não disparam nenhuma regra**. Essa limitação e a forma de corrigi-la estão na Seção 8.

2 Requisitos da Atividade e Atendimento

Requisito	**Como foi atendido**	**Situação**
Entrevista dinâmica	5 dimensões: afinidade matemática, preferência diária, linguagem, entrega de valor e aspecto técnico	Atendido
Motor Experta com 20 a 25 regras	35 regras `@Rule` cruzadas em `src/engine.py`	Atendido
Justificativa transparente	Interface lista a trilha, a pontuação e as regras disparadas	Atendido
Interface web com Streamlit	`app.py` com formulário e exibição do diagnóstico	Atendido
Matriz de decisão (Fase 1)	`docs/MATRIZ_DECISAO.md`	Atendido
Pelo menos 4 personas	10 personas em `tests/personas.py`, uma por trilha	Atendido
README e GitHub	`README.md` com instalação e execução; repositório publicado no GitHub	Atendido

3 Tecnologias Utilizadas

Camada	Tecnologia	Papel no projeto
Linguagem	Python 3.10 a 3.12	Implementação de todo o sistema
Motor de inferência	Experta	Fatos (`Fact`), regras (`@Rule`) e encadeamento para frente
Interface	Streamlit	Formulário da entrevista e exibição do resultado
Versionamento	Git / GitHub	Controle de versão e entrega do repositório

Observação: o Experta ainda não suporta Python 3.13 ou superior, e a dependência antiga de `frozendict` exige a versão 2.0 ou maior no Python 3.10+, como já registrado no README.

4 Engenharia de Conhecimento

4.1 Fatos de entrada

Cada resposta da entrevista vira um fato do Experta. A quinta dimensão (aspecto técnico) foi acrescentada para separar trilhas próximas, principalmente DevOps e Cloud.

Tabela 1: Dimensões da entrevista

Fato	Pergunta	Valores
`AfinidadeMatematica`	Afinidade com matemática/estatística	baixa, media, alta
`PreferenciaDiaria`	O que mais gosta de fazer	algoritmos, layouts, servidores, dados_brutos, seguranca, qualidade, negocio, pipelines, mobile
`Linguagem`	Linguagem/ferramenta de maior afinidade	python, javascript, linux_redes, nenhuma
`EntregaValor`	Valor que prefere entregar	usuario_final, infraestrutura, seguranca, dados_insights, negocio
`AspectoTecnico`	Aspecto técnico que mais atrai	automacao, arquitetura, descoberta, resolucao, criacao

4.2 Conflitos entre trilhas e como são resolvidos

Conflito	Critério de diferenciação
Ciência de Dados × Eng. de Dados	dados_brutos + afinidade alta + insights → Ciência de Dados; pipelines ou dados_brutos + infraestrutura → Eng. de Dados
Back-end × DevOps	algoritmos + usuario_final → Back-end; servidores + infraestrutura + automação → DevOps
DevOps × Cloud	aspecto automacao → DevOps; aspecto arquitetura → Cloud
Front-end × Mobile	layouts + javascript → Front-end; preferência mobile → Mobile
QA × Back-end	qualidade como preferência → QA; algoritmos como preferência → Back-end
Gestão de Produtos	negocio como preferência combinado com entrega de negócio

5 Arquitetura e Implementação

5.1 Estrutura do sistema

A interface coleta as respostas e chama o método `diagnosticar`. Ele reinicia o motor, declara os cinco fatos, executa as regras e devolve a trilha vencedora com a justificativa.

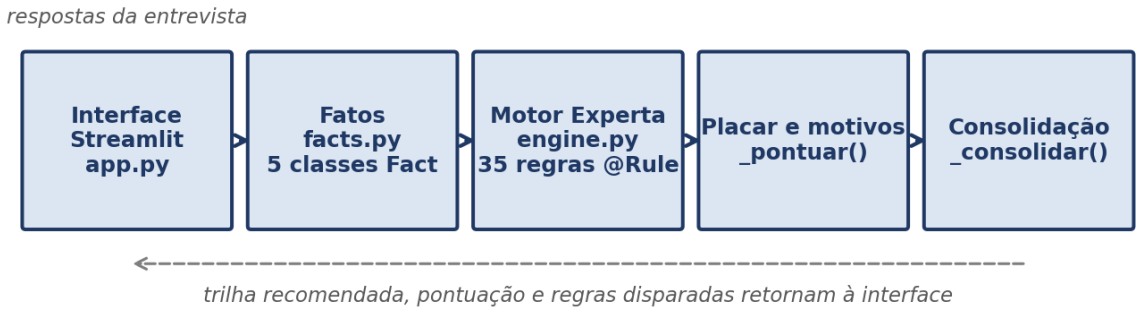


Figura 1: Fluxo de dados entre os módulos

Arquivo	**Responsabilidade**
app.py	Interface Streamlit: formulário, chamada ao motor e exibição do resultado
src/facts.py	Cinco classes Fact, uma por dimensão da entrevista
src/engine.py	Classe ConsultorCarreiras: 35 regras, pontuação e consolidação
src/trilhas.py	Catálogo das 10 trilhas, opções válidas e rótulos amigáveis
tests/personas.py	10 personas de validação executadas sem interface

5.2 Base de regras

Cada regra combina de uma a três condições e, ao disparar, chama _pontuar, que soma pontos à trilha e registra o motivo em texto. A Figura 2 mostra um exemplo, e a Tabela 2 resume a distribuição.

```
@Rule(PreferenciaDiaria(tipo="servidores"),
      AspectoTecnico(aspecto="arquitetura"),
      salience=10)
def regra_cloud_servidores_arquitetura(self):
    self._pontuar("cloud", 5,
        "Servidores + projeção de arquiteturas escaláveis = "
        "Cloud Architecture (AWS/Azure/GCP).")
```

Figura 2: Exemplo de regra (trilha Cloud)

Tabela 2: Regras por trilha

Trilha	Regras	Pontos máx.	Exemplo (saliência → pontos)
Back-end	4	16	algoritmos + python (10 → 5)
Front-end e UI/UX	3	12	layouts + javascript (10 → 5)
Ciência de Dados e ML	4	18	dados_brutos + alta + python (12 → 6)
Engenharia de Dados	5	20	pipelines + python (11 → 5)
Cibersegurança	3	10	seguranca (10 → 4)
DevOps e SRE	4	15	servidores + automacao (10 → 5)
Cloud	3	13	servidores + arquitetura (10 → 5)
Mobile	3	10	mobile + javascript (10 → 5)
QA	3	11	qualidade + resolucao (10 → 5)
Gestão de Produtos	3	13	negocio + entrega negocio (10 → 6)
Total	**35**		

5.3 Pontuação, consolidação e desempate

Depois da execução, `_consolidar` escolhe a trilha de maior pontuação. Em caso de empate, vale o maior número de regras disparadas e, persistindo o empate, a ordem do catálogo (`ORDEM_DESEMPATE`). Se nenhuma regra disparar, o sistema devolve uma mensagem pedindo que a entrevista seja refeita.

Sobre a saliência. Como a pontuação é uma soma, a ordem de disparo não muda o resultado. A `salience` define só a sequência em que as regras são executadas e registradas, não a trilha vencedora.

6 Interface Web

O formulário reúne as cinco perguntas em duas colunas, com botões de opção para afinidade e entrega e listas suspensas para os demais campos. Ao clicar em "Diagnosticar trilha", a página mostra:

- a trilha recomendada e sua descrição;
- a pontuação acumulada;
- a justificativa, com cada regra disparada e seus pontos;
- um painel expansível com o placar de todas as trilhas que pontuaram;
- um aviso quando nenhuma regra é disparada.

7 Validação

7.1 Personas de teste

Foram criadas dez personas, uma por trilha. O script `tests/personas.py` roda o motor sem a interface e compara a trilha obtida com a esperada. Todas as dez coincidiram.

Tabela 3: Resultado por persona (afinidade / preferência / linguagem / entrega / aspecto)

Persona	Respostas	Trilha obtida	Pontos	Regras	Resultado
Ana	alta / dados_brutos / python / dados_insights / descoberta	Ciência de Dados	18	4	OK
Bruno	media / servidores / linux_redes / infraestrutura / automacao	DevOps e SRE	15	4	OK
Carla	baixa / layouts / javascript / usuario_final / criacao	Front-end	12	3	OK
Diego	baixa / negocio / nenhuma / negocio / criacao	Gestão de Produtos	13	3	OK
Eva	media / servidores / linux_redes / infraestrutura / arquitetura	Cloud	13	4	OK
Fábio	media / dados_brutos / python / infraestrutura / automacao	Engenharia de Dados	7	2	OK
Gabriela	media / seguranca / linux_redes / seguranca / resolucao	Cibersegurança	10	3	OK
Henrique	media / algoritmos / python / usuario_final / arquitetura	Back-end	12	3	OK
Isabela	media / mobile / javascript / usuario_final / criacao	Mobile	10	3	OK
João	baixa / qualidade / nenhuma / usuario_final / resolucao	QA	11	3	OK

Os pares ambíguos foram separados como previsto. Ana e Fábio compartilham dados brutos e Python, mas diferem em afinidade, entrega e aspecto, e caem em Ciência de Dados e Engenharia de Dados. Bruno e Eva compartilham servidores, Linux e infraestrutura, e só o aspecto técnico (automação ou arquitetura) os separa. No caso de Eva, DevOps chegou a pontuar 2 pontos pela regra genérica de Linux/Redes, contra 13 de Cloud.

7.2 Cobertura de todo o espaço de respostas

Além das personas, foram testadas as 2.700 combinações possíveis de respostas ($3 \times 9 \times 4 \times 5 \times 5$). Em 2.058 (76,2%) o sistema recomenda uma trilha, sendo 13 casos de empate resolvidos pelo critério da Seção 5.3. Nas outras 642 (23,8%), nenhuma regra dispara.

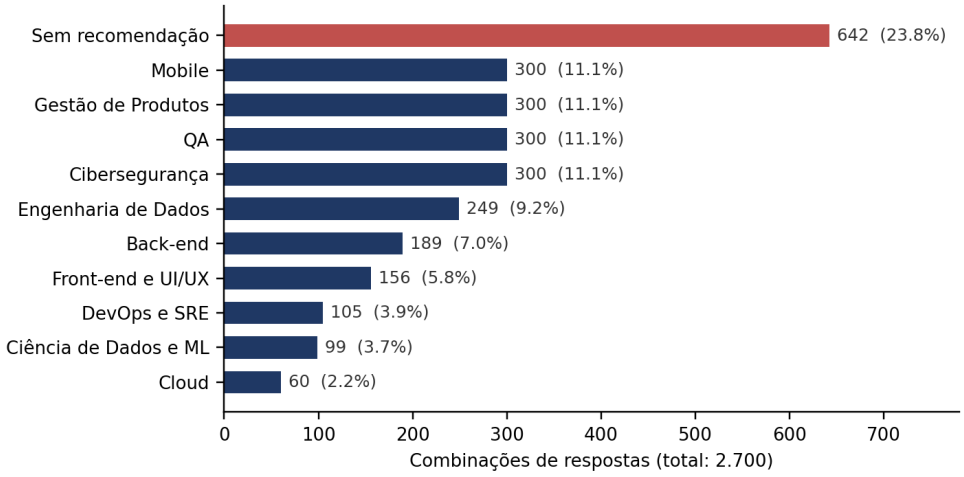


Figura 3: Trilha recomendada nas 2.700 combinações de respostas

8 Limitações e Melhorias

Limitação	**Detalhe**
Combinações sem diagnóstico	As preferências layouts (144), pipelines (135), servidores (126), algoritmos (120) e dados_brutos (117) não têm regra de uma só condição. Seguranca, qualidade, negocio e mobile têm e sempre geram recomendação.
Cobertura desigual	Trilhas com regra genérica vencem em 300 combinações cada; Cloud vence em apenas 60.
Dimensões pouco usadas	A afinidade matemática aparece em só 3 regras, e a linguagem "nenhuma" nunca pontua.
Placar pouco comparativo	Com regras esparsas, geralmente só uma ou duas trilhas pontuam.

Melhorias sugeridas: incluir regras de uma condição (*fallback*) para as cinco preferências sem cobertura total, usar mais a afinidade e a linguagem nas regras, exibir as duas ou três trilhas mais próximas e adicionar ao repositório um teste automatizado de cobertura como o da Seção 7.2.

9 Como Executar

```
python -m venv .venv
source .venv/bin/activate        # Windows: .venv\Scripts\activate
pip install -r requirements.txt
```

```
streamlit run app.py              # interface web (localhost:8501)
python -m tests.personas          # validação: 10/10 personas corretas
```

10 Conclusão

O projeto cumpre os requisitos da atividade: motor Experta com 35 regras cruzadas, justificativa transparente, interface em Streamlit e validação com dez personas, todas corretamente classificadas. A varredura de cobertura mostrou que cerca de um quarto das combinações ainda não gera recomendação, o que pode ser corrigido com regras genéricas adicionais. Por ser baseado em regras, o sistema permite acrescentá-las e auditá-las de forma independente.